

Automated License Plate Recognition (ALPR) System: The Build

From Blueprint to Static Access Control Prototype

Course: ITAI 1378 — Computer Vision & AI
Scope: Tier One (+ Tier Two Full-Stack OCR/DB Extension)

Project Presented by:

Oluwatoyin Fadiran (*Tier One*)

LICENSE
PLATE
DETECTION

The Problem: Manual Patrols vs. Automated Access Control

1. THE PROBLEM

Manual license plate logging during security rounds is slow, tedious, and introduces critical human error into logs.

2. THE IMPACT

Wastes active patrol time, leaves parking perimeters unmonitored, and delays responses to unauthorized vehicles.

3. THE AUTOMATION WIN

A deployed CV pipeline that automatically detects plates, extracts ASCII text, and cross-checks a database instantly.

What We Built: End-to-End Enterprise ALPR Pipeline

The One-Sentence Solution:

We built an automated CV pipeline using YOLO11n for spatial detection, PaddleOCR-GPU for ASCII character extraction, and SQLite for real-time security alerting.

1. YOLO Localization



ACTUAL PRODUCTION WORKFLOW

STAGE 1: 01_detection.ipynb

- Input: Raw Parking Lot Images (/LPR_Inbox/incoming)
- Process: Load best.pt (YOLO11n) -> Detect Plates -> Save Crops
- Output: Cropped plates (.jpg) & JSON spatial coordinates

STAGE 2: 02_ocr_database.ipynb

- Process: PaddleOCR-GPU (Deep Learning Neural Network)
- Clean: Sanitize Text (Strip US State names like 'ALABAMA')
- Output: Query SQLite (lpr.db) -> Auth Status (✅ / 🚨)

2. Extracted Crop



OCR Output: GNX3211

Confidence: 0.94

Database Lookup:

Table: authorized_vehicles

Match: FOUND

Final Status:

☐ AUTHORIZED

Live System Demonstration: End-to-End Execution

 [[Watch Project Walkthrough Video](#)]

Live Walkthrough of 01_detection.ipynb & 02_ocr_database.ipynb

- Demonstrates real vehicle images -> YOLO localization -> SQLite lookup
 - Includes Honest Engineering Analysis of at least one failure case (inability to avoid False Positive License Plates fully)

How It Works: Computer Vision Technical Approach

Architectural Layer	Stage 1: Spatial Detection	Stage 2: Recognition & Access
CV Technique	Single-Stage Object Detection	Deep Learning OCR + SQL Relational Lookup
Model Architecture	YOLO11n (Nano)	PaddleOCR-GPU (Deep Learning NN)
Model Usage	Pretrained + Fine-Tuned (Transfer Learning)	Pretrained GPU NN + Custom Sanitization Loop
Framework & Stack	PyTorch + Ultralytics YOLO	PaddlePaddle / Python / SQLite (lpr.db)
Why Built This Way	Lightweight, high-speed spatial inference.	Scrapped EasyOCR for GPU NN accuracy.

The Data We Used: Provenance & Preprocessing

1. DATASET PROVENANCE

Sourced from Roboflow Universe
(Project: rxg4e, v13).

- Volume: 10,125 high-quality images
- Split: 70% Train / 20% Valid / 10% Test
- Label: Single bounding-box class ('license-plate').

2. LABELING & SANITIZATION

Pre-labeled in YOLO txt syntax.
We applied programmatic Python
cleaning:

- Dynamic multi-plate image cropping
- Stripping 50 normalized US State names (e.g., 'TEXAS') before SQL queries.

3. SIT / UAT EVALUATION

To test system generalization beyond the training distribution, custom real-world parking lot vehicle images were added to evaluate deployment readiness.

Results vs. Our Promise I: Blueprint Targets Exceeded (Tabular)

Evaluation Metric	Blueprint Promised Target	Validation Split	Test Split	Status
Primary: mAP@0.50	>= 0.9000	0.9686	0.9695	✅ PASSED
mAP@0.50-0.95	Baseline Reference	0.6913	0.6912	Evaluated
Precision / Recall	Baseline Reference	0.9825 / 0.9473	0.9903 / 0.9456	Evaluated
Secondary: Total Latency	< 0.2s (200.0 ms) per image	11.3 ms total	11.3 ms total	✅ PASSED
Core Inference Speed	< 0.2s (200.0 ms) per image	4.2 ms / image	4.2 ms / image	47x FASTER

HONEST ENGINEERING TAKEAWAY

Our model operates at 4.2ms per image (~47x faster than our real-time baseline) while achieving a ~0.97 mAP on completely unseen test datasets. Both promised Blueprint metrics were comfortably achieved.

Results vs. Our Promise II: Blueprint Targets Exceeded (Visual)

ALPR FINAL PROJECT — TIER ONE BUILD

MODEL METRICS SUMMARY

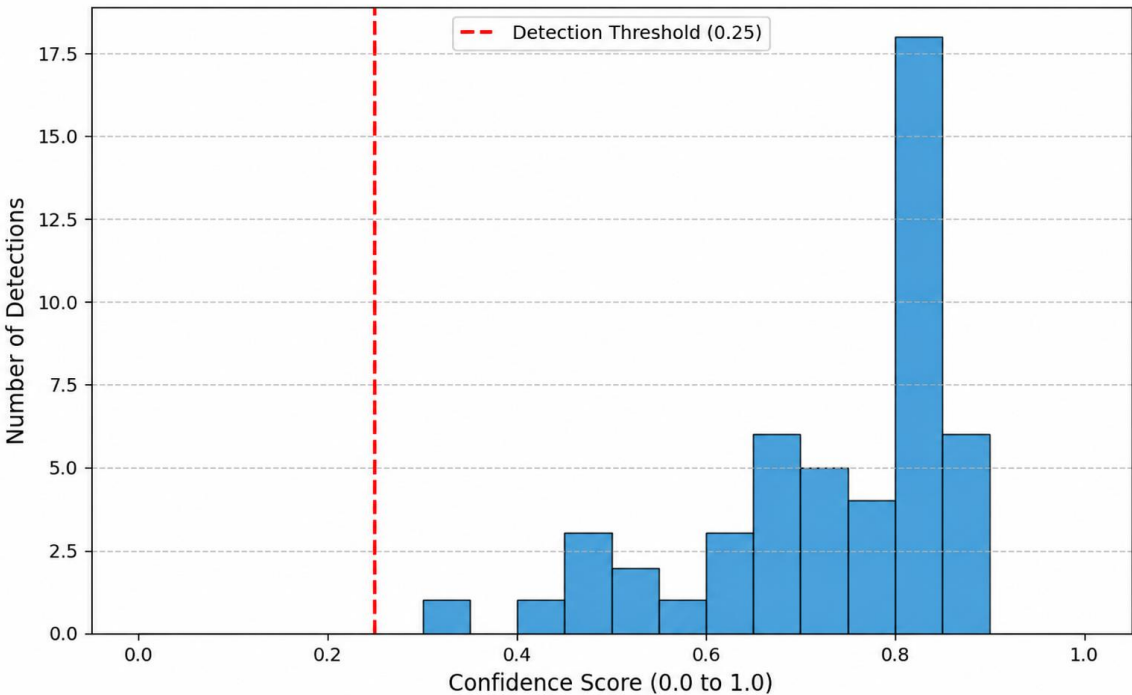
Total Plates Detected: 50

Average Confidence: 0.74

OUTLIERS (Confidence < 0.5):

	image	confidence
35	PXL_20260728_201351991.jpg	0.3230
34	PXL_20260728_201351991.jpg	0.4226
1	test1_final.jpg	0.4612
33	PXL_20260728_201351991.jpg	0.4655
16	PXL_20260728_202346206.MP.jpg	0.4687

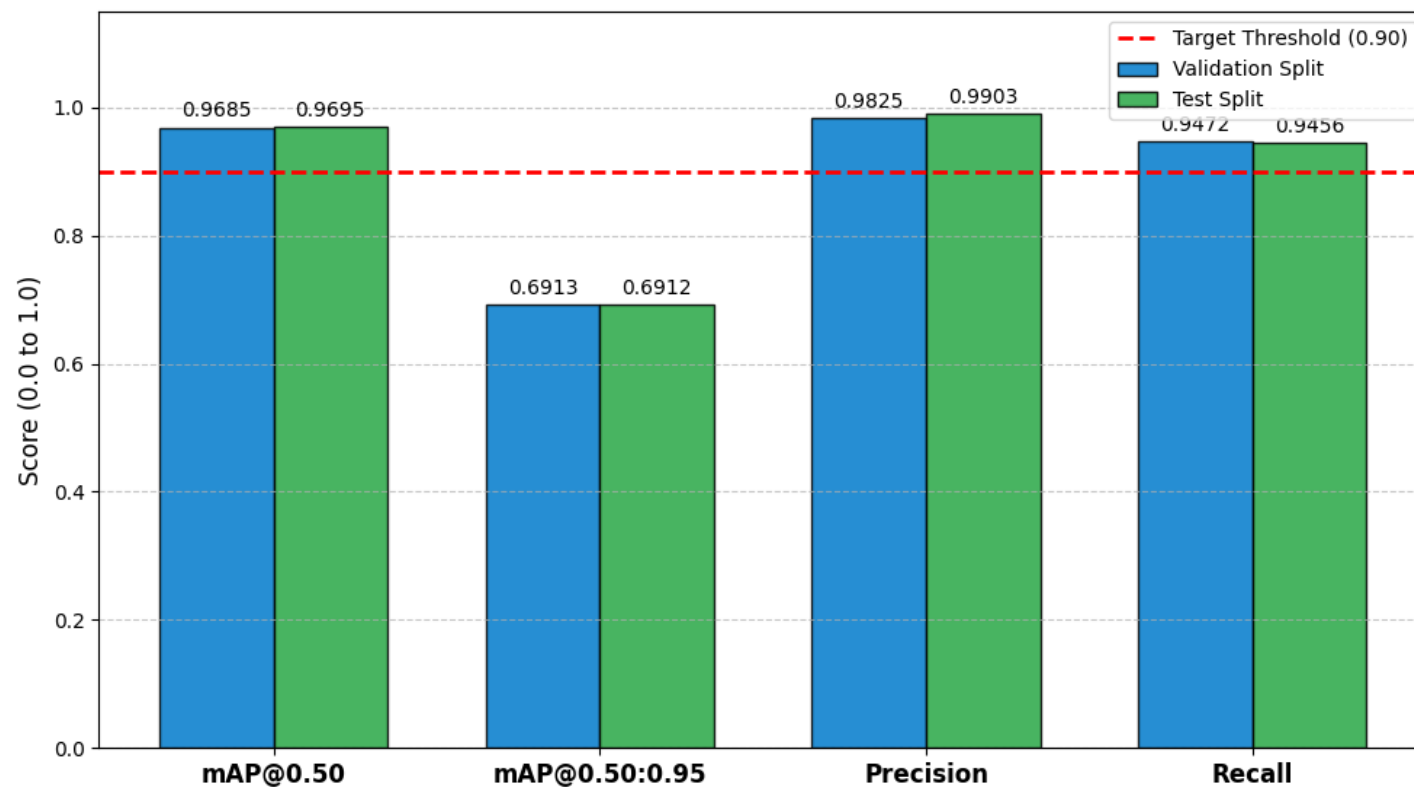
YOLOv11 Detection Confidence Distribution



Results vs. Our Promise III: CV Data (Visual Metics)

ALPR FINAL PROJECT — TIER ONE BUILD

YOLOv11 Performance: Validation vs. Test



What Changed From Blueprint: Architectural Pivots

Dimension	Original Blueprint Plan	Actual Build Pivot	Engineering Rationale (The 'WHY')
OCR Engine	EasyOCR (CPU Bound)	PaddleOCR-GPU	EasyOCR consistently misrecognized alphanumeric characters.
Proposed Fix	OpenCV Image Preprocessing	Engine Replacement	Resizing/contrast failed; issue was model capacity, not contrast.
Hardware Stack	Standard CPU Inference	NVIDIA T4 GPU (Colab)	PaddleOCR-GPU uses a deeper neural network requiring GPU math.
What Was Kept?	YOLO Plate Detection	Unchanged	YOLO11n localization & SQLite lookup worked exactly as planned.

Challenges & Fixes: Managing Engineering Constraints

CHALLENGE 1: OCR ACCURACY

- Problem: CPU-bound EasyOCR consistently misread characters on real license plate crops.
- Failed Attempt: Countless OpenCV preprocessing loops (grayscale, upscaling, thresholding) failed to fix misreads.
- Engineered Fix: Swapped the engine for a GPU-grade neural network (PaddleOCR-GPU), immediately resolving accuracy issues.

CHALLENGE 2: CLOUD TIMEOUTS

- Problem: Training YOLO11n for 100 epochs on 10,125 images hit Google Colab's free-tier GPU limits (disconnecting after ~2 hours).
- Engineered Fix: Optimized training hyperparameters by scaling down to epochs=50 (batch=16, imgsz=640).
- Outcome: Successfully converged without timeouts while maintaining ~0.97 mAP spatial accuracy.

Lessons Learned & Next Steps for Architecture

WHAT I LEARNED

- Full-Stack CV Coupling: Learned how to build an end-to-end POC pipeline that bridges bounding-box spatial detection with character recognition.
- OCR Preprocessing Limits: Discovered that model architecture capacity often matters more than input image pre-filtering.
- SQL Integration: Built an automated relational logging table to track access authorization.

NEXT STEPS: REAL-TIME VIDEO

- Video Stream Capture: Transition from batch static images to an RTSP or MP4 video ingestion pipeline.
- Why Video?: YOLO11 is inherently optimized for high-FPS video inference.
- Security Advantage: Allows continuous vehicle tracking, capturing multiple plate angles before a driver even parks.